DEEP DIVE · 20 SLIDES

Concurrency Control in Distributed Systems

A complete breakdown of how distributed systems handle concurrent data access - from race conditions to transactions, lock types, isolation levels, and locking strategies.

Race Conditions | ACID Transactions | Lock Types | Isolation Levels | Optimistic Locking | Pessimistic Locking | Spring Data JPA | Decision Guide

Whether you are preparing for a FAANG interview or building production microservices - mastering concurrency is non-negotiable. Save this post.

PART 1 - THE PROBLEM

Why Concurrency Breaks Things

When multiple users or processes try to access and modify a shared resource **simultaneously**, conflicts arise. The result is corrupted, inconsistent data - and in production, it can mean real financial loss or duplicate records.

CLASSIC EXAMPLE - MOVIE SEAT BOOKING RACE CONDITION

User A - Instance 1		User B - Instance 2
SELECT status WHERE seat=101		SELECT status WHERE seat=101
Result: 'available' ✓		Result: 'available' ✓
Business logic - proceed to book		Business logic - proceed to book
UPDATE seat SET status='booked'	⚡	UPDATE seat SET status='booked'
COMMIT - Booking confirmed		COMMIT - Also "confirmed"!

Both reads happened BEFORE either write completed. One seat, two confirmed bookings. Data is corrupt.

The Goal: Ensure data integrity and prevent race conditions across all concurrent operations.

The Challenge: In a distributed microservices architecture, traditional single-process solutions do not work.

This class of bug is called a **race condition** - where the outcome depends on the unpredictable timing of concurrent operations. At scale, with hundreds of requests per second, these bugs become near-certain occurrences, not edge cases.

PART 2 - LOCAL LOCKING

Why Local Locks Always Fail

Developers often reach for **synchronized blocks** first. This only works within a single JVM process - and completely breaks the moment you deploy more than one service instance.

Java

```
public synchronized void bookSeat(int seatId) {
    // Only safe within ONE JVM instance!
    // Completely invisible to all other service instances.
    // This gives you a false sense of safety at scale.
}
```

WHY SYNCHRONIZED BREAKS IN DISTRIBUTED SYSTEMS

Instance 1 (JVM process #1)

🔒 Lock held in memory

User A is inside the synchronized block. Lock lives in this JVM's heap memory only.

Instance 2 (JVM process #2)

🔓 No lock visible here

User B proceeds freely. JVM #2 has no knowledge of the lock held in JVM #1's memory.

Instance 1 → Shared DB Row ← Instance 2

Both instances point to the SAME database row. Both modify it. Data corruption occurs.

Bottom line: Once you scale to multiple service instances, local JVM locks are worthless. The database is the single shared resource - and it must enforce all the rules.

- Each JVM process has its **own heap memory** - completely isolated from other processes.
- A synchronized lock in Instance 1 is **invisible** to Instance 2 - they have zero shared memory.
- The only way to coordinate across instances is through a **shared external resource** - the database.

Database Transactions

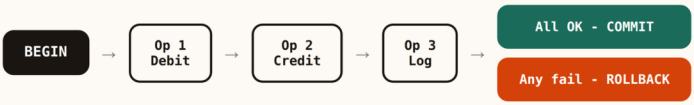
A transaction groups multiple DB operations into a **single atomic unit of work**. Either everything succeeds, or nothing does. There are no partial states - ever.

SQL - Money Transfer Transaction

```sql
BEGIN TRANSACTION;

-- Step 1: Debit $500 from sender
UPDATE accounts SET balance = balance - 500
WHERE id = 1;

-- Step 2: Credit $500 to recipient
UPDATE accounts SET balance = balance + 500
WHERE id = 2;

-- Both succeed: persist changes
COMMIT;

-- Or: if anything fails, undo everything
-- ROLLBACK;
```

Why this matters: If the credit step fails after the debit already ran, the database automatically rolls back the debit too. Money is never lost. No manual cleanup needed. The database stays consistent regardless of what fails.

TRANSACTION FLOW



Transactions are the foundation that everything else - locking, isolation levels, concurrency control - is built on top of. Without the transactional guarantee, no amount of application-level logic can ensure correctness.

PART 3B - ACID PROPERTIES

ACID Properties Explained

Every database transaction must satisfy four properties that together guarantee correctness - even under failure, crashes, or high concurrency. ACID is not optional in financial or booking systems.

A Atomicity

All operations in a transaction **succeed or none do**. There are no partial updates, ever. If the system crashes halfway through a multi-step operation, the database rolls everything back automatically - leaving no broken intermediate state behind.

C Consistency

The database moves from one **valid state to another valid state**. Business rules and constraints are always enforced. A transaction can never leave data in a broken or invalid condition - foreign keys, check constraints, and triggers all remain satisfied.

I Isolation

Concurrent transactions **do not interfere** with each other. The intermediate state of a transaction is invisible to others until committed. Users always see clean, consistent data - regardless of what other transactions are doing simultaneously.

D Durability

Once a transaction is committed, data is **permanently persisted to disk**. A power outage, crash, or system failure cannot erase a committed write. The database write-ahead log (WAL) ensures recovery to the last committed state.

Interview tip: When asked "how do databases ensure data integrity?", always lead with ACID. Then walk through each property with a concrete example. Interviewers love specific examples over abstract definitions.

PART 4 - LOCK TYPES

Database Lock Types

Locks control row and table access between concurrent transactions. There are two fundamental types - and understanding them is essential for reasoning about any concurrency scenario.

S-LOCK Read Lock

Shared Lock

Multiple transactions can hold a shared lock **simultaneously**. All of them can read the data freely. No writes are allowed while any shared lock is active on that row. Acquired automatically by SELECT in some isolation levels.

```
SELECT * FROM seats WHERE id = 101;
```

X-LOCK Write Lock

Exclusive Lock

Only **one transaction** can hold an exclusive lock at a time. No other transaction can read or write the locked row until the lock is released at COMMIT or ROLLBACK. Must be explicitly requested.

```
SELECT * FROM seats WHERE id=101 FOR UPDATE;
```

LOCK COMPATIBILITY MATRIX

	S-Lock Held	X-Lock Held
New S-Lock	Compatible Both can read	Conflict Must wait
New X-Lock	Conflict Must wait	Conflict One writer only

Rule: Multiple readers OR one exclusive writer - NEVER both at the same time.

Locks are held for the **duration of the transaction** (until COMMIT or ROLLBACK), not just for the duration of the individual SQL statement. This is a common source of confusion - a lock acquired mid-transaction stays active.

PART 5A - READ PROBLEMS

Dirty Read Explained

Definition: Transaction A reads uncommitted data from Transaction B. If B rolls back, A has acted on data that **never actually existed** in a committed state in the database.

DIRTY READ - FULL TIMELINE

```
T=1 T1   BEGIN
T=2 T1   UPDATE seat SET status='booked' -- NOT committed yet
T=3 T2   BEGIN
T=4 T2   SELECT status -- reads 'booked' (DIRTY READ of uncommitted data)
T=5 T2   Sends booking confirmation email based on 'booked' status...
T=6 T1   ROLLBACK -- payment failed. The 'booked' write is undone. Never committed.
T=7 T2   Now in corrupt state. Email sent for booking that does not exist.
```

Key danger: T2 made a real-world decision based on data that was never permanently saved. The confirmation email went out, the ticket was issued - but the underlying database state was rolled back to "available". Pure phantom data.

Protection: Read Committed isolation level and above prevent dirty reads. You only ever see committed data.

Real-world impact: In financial systems, dirty reads can cause overdrafts, double charges, and regulatory violations.

PART 5B - READ PROBLEMS

Non-Repeatable Read Explained

Definition: Transaction A reads a row, then reads the **same row again**. Between the two reads, Transaction B has updated and committed that row. A gets different values for the same query within the same transaction.

NON-REPEATABLE READ - FULL TIMELINE

```
T=1 T1  BEGIN
T=2 T1  SELECT price FROM product WHERE id=1 -- returns $100
T=3 T1  Calculates 10% discount = $10. Prepares invoice for $90...
T=4 T2  UPDATE product SET price=150 WHERE id=1; COMMIT;
T=5 T1  SELECT price FROM product WHERE id=1 -- now returns $150!
T=6 T1  Invoice is wrong. Read $100, computed discount, actual price is $150.
```

Key problem: T1 expected to get the **same value** it read before in the same transaction. It got something different. Any business logic that assumes a value will not change mid-transaction is now producing incorrect results.

Protection: Repeatable Read isolation level prevents this. Shared locks are held on read rows until the transaction ends.

Difference from Dirty Read: The data T2 wrote IS committed. The problem is T1 seeing different values for the same row within one transaction.

This anomaly is particularly dangerous in **reporting and invoice generation** systems where values read at the start of a calculation must remain consistent throughout the entire operation.

PART 5C - READ PROBLEMS

Phantom Read Explained

Definition: Transaction A runs a range query, then runs the **same range query again**. Between the two reads, Transaction B inserted new rows matching the condition. A sees "ghost rows" that were not there before.

PHANTOM READ - FULL TIMELINE

```
T=1 T1  BEGIN
T=2 T1  SELECT * FROM seats WHERE status='available' -- returns 5 rows
T=3 T1  Logic: Only 5 seats left. Send low availability alert to users...
T=4 T2  INSERT INTO seats (status) VALUES ('available'); COMMIT;
T=5 T1  SELECT * FROM seats WHERE status='available' -- now returns 6 rows!
T=6 T1  Alert was wrong. A phantom row appeared mid-transaction.
```

Why phantom reads are different: Unlike non-repeatable reads (which affect existing rows), phantom reads involve **new rows appearing** in a range query result. Shared row locks cannot prevent this - you need range locks that block INSERTs into the matching range.

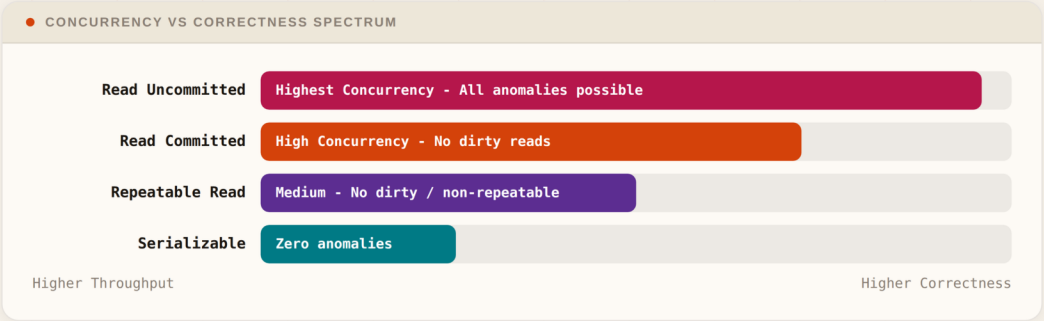
Protection: Only Serializable isolation prevents phantoms. It uses range locks (predicate locks) to block any INSERT into the queried range.

Common scenario: Counting available resources, checking capacity limits, enforcing business rules based on row counts.

PART 6 - ISOLATION LEVELS

The 4 Isolation Levels

Isolation levels let you tune the trade-off between **data correctness** and **system concurrency**. Higher isolation means fewer anomalies but more locking, more contention, and lower throughput.



PostgreSQL default: Read Committed. Shared lock acquired and released immediately after each read. Good enough for most web applications.

MySQL InnoDB default: Repeatable Read. Shared lock held until transaction ends. Same row always returns same value within one transaction.

Choosing the wrong isolation level is one of the most common production bugs. Running at Read Uncommitted for "performance" in a banking system is catastrophic. Running Serializable for a read-heavy analytics dashboard wastes throughput.

PART 6 - FULL BREAKDOWN

Isolation Levels - Complete Reference

LEVEL	DIRTY READ	NON-REPEATABLE	PHANTOM	USE WHEN
`Read Uncommitted`	Possible	Possible	Possible	Analytics, approx counts only
`Read Committed`	Prevented	Possible	Possible	Most web apps - PG default
`Repeatable Read`	Prevented	Prevented	Possible	Reports, invoicing - MySQL default
`Serializable`	Prevented	Prevented	Prevented	Finance, seat booking, critical ops

Read Uncommitted

No locks acquired for reads. Every anomaly is possible. **Never use in production** unless you genuinely only need approximate counts - and even then, think twice. No real-world transactional system should use this.

Serializable

Range locks prevent phantom rows entirely. No read anomalies of any kind. This is the **slowest isolation level**. Use it for critical financial operations, seat booking, and any scenario where data correctness is non-negotiable and wrong answers are worse than slow answers.

In Spring: `@Transactional(isolation = Isolation.SERIALIZABLE)` - set at method level on your service class.

PART 7 - OPTIMISTIC LOCKING

Optimistic Concurrency Control (OCC)

Philosophy: Conflicts are **RARE**. Do not lock the row. Let multiple users read and work freely. Only at save time, check if anyone else changed the data. If yes, retry.

The key mechanism is a **version column** added to every table that needs optimistic locking. Every successful UPDATE increments the version number. At save time, the WHERE clause checks that the version you read is still the current version in the database.

SQL - Version Column Schema

```sql
CREATE TABLE seats (
    id      INT PRIMARY KEY,
    status  VARCHAR(20),
    version INT DEFAULT 0   -- The optimistic lock version
);

-- Step 1: Read row + current version
SELECT id, status, version FROM seats WHERE id = 101;
-- Returns: id=101, status='available', version=1

-- Step 2: Update with version check in WHERE clause
UPDATE seats
SET    status = 'booked', version = 2
WHERE  id = 101 AND version = 1; -- Must still be version 1

-- rows_affected = 1: success, you got there first
-- rows_affected = 0: conflict, someone changed it, retry
```

Why this works: If two users both read version=1 and both try to update, only the first one's update succeeds (version 1 matches). The second user's update finds version=2 now and gets rows_affected=0. It must retry from the beginning - re-reading the latest version.

PART 7 - OCC FLOW & JPA

OCC Flow & JPA Implementation

OPTIMISTIC LOCKING - STEP BY STEP FLOW

1. READ — Read row and current version (e.g. version = 1). No lock held.

2. WORK — Execute business logic freely. Zero blocking of other users.

3. WRITE — UPDATE ... WHERE id=101 AND version=1

rows_affected = 1
Success. You got there first. COMMIT.

rows_affected = 0
Version changed. Conflict. Retry from Step 1.

Java - Spring Data JPA

```java
// Entity - just add @Version
@Entity
public class Seat {
    @Id
    private Long id;
    private String status;

    @Version   // JPA auto-increments and checks version
    private Integer version;
}

// Service - catch conflict, retry
@Transactional
public void bookSeat(Long seatId) {
    Seat seat = seatRepository.findById(seatId).orElseThrow();
    seat.setStatus("booked");
    try {
        seatRepository.save(seat); // JPA adds WHERE version=N automatically
    } catch (OptimisticLockingFailureException e) {
        // Version mismatch - retry the entire operation
        bookSeat(seatId); // Or use retry framework
    }
}
```

PART 8 - PESSIMISTIC LOCKING

Pessimistic Concurrency Control (PCC)

Philosophy: Conflicts are **LIKELY**. Lock the row immediately when you read it. No one else can read or write it until you commit. Pay the blocking cost upfront.

PCC uses **SELECT FOR UPDATE** to acquire an exclusive lock the moment the row is read. Any other transaction trying to touch that row will **block and wait** until the lock holder commits or rolls back. No version columns needed - the database enforces it.

SQL - Pessimistic Locking

```sql
BEGIN;

-- Acquires exclusive lock IMMEDIATELY at read time
-- All other transactions trying to touch row 101 will BLOCK here
SELECT * FROM seats WHERE id = 101 FOR UPDATE;

-- You are the ONLY transaction with access to this row
-- Safe to read, compute, and update with no race condition
UPDATE seats SET status = 'booked' WHERE id = 101;

-- Lock is released here. Blocked transactions can now proceed.
COMMIT;
```

Deadlock Warning: If two transactions each hold a lock and wait for the other's lock, neither can proceed. The database detects this cycle and kills one transaction with a DeadlockLoserDataAccessException. You must always handle this with retry logic.

PART 8 - PCC FLOW & SPRING

PCC Flow & Spring Data Implementation

PESSIMISTIC LOCKING - FULL TRANSACTION TIMELINE

T=1 T1	**SELECT ... FOR UPDATE - exclusive lock acquired on row 101**
T=2 T2	*SELECT ... FOR UPDATE on row 101 - BLOCKED. Waiting for T1 to finish.*
T=3 T1	Business logic runs safely. No race condition possible.
T=4 T1	UPDATE seats SET status='booked' WHERE id=101
T=5 T1	**COMMIT - lock released. T2 is now unblocked.**
T=6 T2	**T2 reads row - now sees status='booked' - handles gracefully.**

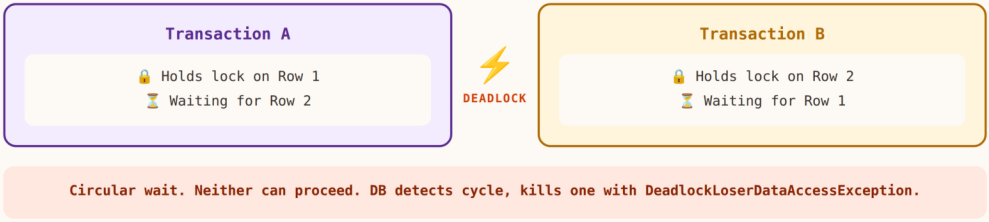
Java - Spring Data JPA

```java
// Repository - acquire exclusive lock at read time
@Lock(LockModeType.PESSIMISTIC_WRITE)
@Query("SELECT s FROM Seat s WHERE s.id = :id")
Optional<Seat> findByIdWithLock(@Param("id") Long id);

// Service - must handle DeadlockLoserDataAccessException
@Transactional
public void bookSeat(Long seatId) {
    try {
        Seat seat = seatRepo.findByIdWithLock(seatId).orElseThrow();
        if ("available".equals(seat.getStatus())) {
            seat.setStatus("booked");
            seatRepo.save(seat);
        }
    } catch (DeadlockLoserDataAccessException e) {
        // Deadlock detected - retry the transaction
        bookSeat(seatId);
    }
}
```

PART 8 - DEADLOCK DEEP DIVE

Deadlock - What, Why & How to Handle

DEADLOCK SCENARIO - CIRCULAR DEPENDENCY



Circular wait. Neither can proceed. DB detects cycle, kills one with DeadlockLoserDataAccessException.

4 prevention strategies to implement today:

- **Consistent lock ordering** - always acquire locks on resources in the same order across all transactions. If T1 always locks Row 1 before Row 2, and T2 does the same, circular waits become impossible.
- **Keep transactions short** - the less time a lock is held, the lower the window for deadlock to occur. Never do external API calls inside a transaction that holds locks.
- **Retry on deadlock** - always catch DeadlockLoserDataAccessException and retry the entire transaction from scratch. The database already resolved the conflict for you.
- **Use optimistic locking** - if your conflict rate is actually low, switching to OCC eliminates deadlock risk entirely because no rows are ever locked during execution.

PART 9 - COMPARISON

OCC vs PCC Side by Side

🔓 Optimistic (OCC)	🔒 Pessimistic (PCC)
Assumption: Conflicts are rare	**Assumption:** Conflicts are likely
Mechanism: @Version column, check at write time	**Mechanism:** SELECT FOR UPDATE, blocks at read
Locking: No lock during read or work phase	**Locking:** Exclusive lock held from read to COMMIT
Throughput: Higher - no blocking of readers	**Throughput:** Lower - writers block each other
DB load: Lower - no lock management overhead	**DB load:** Higher - lock tracking and management
Deadlock: Zero risk - no locks held	**Deadlock:** Risk present - must handle and retry
Failure: Requires retry logic on conflict	**Failure:** No optimistic fails - but deadlock possible
Best for: User profiles, product views, read-heavy APIs, low-traffic writes	**Best for:** Seat booking, flash sales, bank transfers, inventory management

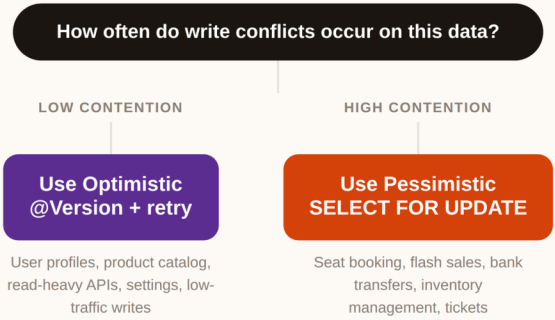
There is no universally "better" approach. OCC shines at scale with low write contention. PCC is the right tool when you cannot afford retries - like in a flash sale where every millisecond counts and overselling is unacceptable.

PART 9 - DECISION GUIDE

How to Choose the Right Strategy

The single most important question: **How frequently do write conflicts actually occur in your workload?** Everything else follows from that answer.

DECISION FLOW



FACTOR	OPTIMISTIC (OCC)	PESSIMISTIC (PCC)
`Conflict frequency`	Low contention	High contention
`Read to write ratio`	Read-heavy	Write-heavy
`Throughput`	Higher	Lower
`Deadlock risk`	None	Yes - handle it
`Retry complexity`	Requires retry logic	No retry for OL fail
`DB lock load`	Lower	Higher (locks held)

KEY TAKEAWAY

The Database Is Your Source of Truth.

In a distributed system, local JVM locks cannot coordinate across instances. Database-level locking - whether optimistic or pessimistic - is the **only reliable way** to prevent race conditions at scale.

- Use **ACID transactions** to group operations atomically - all or nothing, no partial states.
- Choose **Serializable isolation** for critical financial and booking scenarios where data correctness is non-negotiable.
- Use **Optimistic Locking (@Version)** when conflicts are rare and throughput matters more than strict blocking.
- Use **Pessimistic Locking (SELECT FOR UPDATE)** when conflicts are frequent and correctness must be guaranteed without retries.
- Always handle **OptimisticLockingFailureException** and **DeadlockLoserDataAccessException** with proper retry logic. Never swallow these exceptions silently.

If this post helped you understand concurrency control - follow **@AbhishekGoyal** for more Java and System Design deep dives every week.

PART 10 - QUICK REFERENCE

Quick Reference Cheatsheet

🔒 synchronized block

Useless in distributed systems. Only works within one JVM. Invisible to all other service instances. Never use this as your distributed lock.

📖 Shared Lock (S-Lock)

Multiple readers allowed simultaneously. No writes while active on that row. Acquired by SELECT in some isolation levels.

✏️ Exclusive Lock (X-Lock)

One writer only. Blocks all other reads and writes until released. Acquired explicitly via SELECT FOR UPDATE.

⚠️ Read Uncommitted

All three read anomalies are possible. Use only for approximate reads like analytics where correctness is not required.

🌐 Read Committed

No dirty reads. PostgreSQL default. Good for most web applications. Non-repeatable reads still possible.

🔁 Repeatable Read

No dirty or non-repeatable reads. MySQL InnoDB default. Phantom reads still possible in range queries.

🛡️ Serializable

No read anomalies of any kind. Use for critical financial operations and seat booking. Lowest throughput - use deliberately.

🏷️ Optimistic (@Version)

Add @Version to entity. JPA checks version on every save. Catch OptimisticLockingFailureException and retry. Zero deadlock risk.

⛓️ Pessimistic (@Lock)

Add @Lock(PESSIMISTIC_WRITE) to repository method. Row is locked from read to commit. Catch DeadlockLoserDataAccessException and retry.